

Características Gerais do Produto

O datalogger é um equipamento com o objetivo principal de digitalizar sinais elétricos analógicos com uma entrada de sensores tipo universal e gravar esta informação em um cartão do tipo SDHC, podendo o produto funcionar sem a necessidade de um operador ou um notebook.

O produto tem uma dimensão e consumo de energia reduzidos, para que seja possível sua aplicação em locais de difícil acesso e/ou testes de longa duração.

Para utilizá-lo, recomenda-se primeiro configurar o(s) sensor(es) conectado(s) no(s) canal(is) analógico(s), para que se possa obter o melhor sinal possível, isto é, a maior resolução sem distorção do sinal. Depois disso, configura-se o modo de aquisição e gravação e pronto, é só iniciar o ensaio.

Características Gerais das Entradas e Saídas de Sinal

- É possível configurar todo o sistema (tipo de entrada, ganho, balanço, etc) sem precisar abrir o equipamento;
- Sensores: ponte completa, $\frac{1}{4}$ de ponte (120 ou 350), $\frac{1}{2}$ ponte, termopar, corrente, tensão (AC/DC +- 10V), ICP;
- Tensão de excitação de 0 a 5V configurável por canal, com possibilidade de uma outra versão de hardware com tensão de 0 a 10V (maior consumo);
- Alimentação de $\pm 12V$ para sensores;
- Shunt-cal: global ou por canal (interno);
- 8 canais analógicos;
- Filtro analógico anti-aliasing tipo Butterworth fixo em 2048 Hz;
- Frequência de aquisição fixa em 12,5 KS/s;
- Decimação e filtro passa-baixa digital programados por software;
- Ganhos ajustados de fábrica e selecionáveis por software (1,3,10,30,100,300,1000,3000);
- Ganho de usuário programado por software;
- Balanço de $\pm 20mV$ e offset de $\pm 10V$, os dois com 16bits de resolução e selecionáveis por software;
- Impedância de entrada para tensão contínua : 100K Ω ou flutuante, selecionável por software;
- 4 entradas de contadores normais ou 2 duas de contadores tipo quadratura;
- Sensor de 3 eixos de aceleração interno;

Características Gerais das Funções Digitais

- Gravar sinal em cartão tipo SDHC no formato TEM, com velocidade de até 2KHz por canal;
- Gravar dados do GPS em arquivo KML Google Earth (latitude, longitude, velocidade, horário);

- Frequência de amostragem fixa em 12,5KS/s por canal, com possibilidade de redução por decimação;
- Filtro passa-baixa digital;
- Sincronismo por GPS;
- Sincronismo por fio (master,slave);
- Aquisição dos dados via ethernet;
- Download dos dados via web;
- Processamento do sinal adquirido (Rainflow, Análise estatística);
- Triggers de aquisição de dados;

Funcionalidades Gerais Software:

- Configuração do canal analógico
 - auto-zero
 - calibrar com shunt-cal
 - aplicar shunt-cal
 - regressão linear
 - seleção do filtro
 - seleção do ganho
 - seleção da tensão de excitação
 - wizard para escolha de sensor
- Configuração digital
 - Frequência de aquisição
 - GPS
 - Sincronismo
 - Configs de gravação
 - Estados de gravação
 - Relógio

Detalhamento p/ Auto-zero

Para realizar o auto-zero do canal, deve-se atuar no D/A de balanço/offset, utilizando o algoritmo de busca binária, tendo-se como mínimo passo $20\text{mV}/2^{16}$ (caso balanço) ou $20/2^{16}$ (caso offset).

Detalhamento p/ Calibração com shunt-cal

1. obter valor X1 em engenharia do canal a ser calibrado
2. aplicar shunt cal
3. obter valor X2 em engenharia do canal a ser calibrado
4. realizar contas (??) utilizando o valor da variável Canal[n].ValorShunt e colocar valores resultantes na configuração do canal.

Detalhamento p/ Wizard

Seleção do Tipo de sensor:

Strain-Gauge

Configuração do Canal:

¼ de ponte

- fios em pinos 1, 2 e 3.

½ de ponte

- fios em pinos 1, 2 e 3.

Ponte completa

- fios em pinos 1,6 e 2,7.

Tensão de excitação:

- 5V (frisar que é apenas para 350Ω)
- 2,5V
- 1V

Amplificação do Sinal (Ganho), Calibração e ajuste (balanço)

- fundo de escala e sensibilidade -> cálculo de valor de ganho (ganho de usuário) -> faixa de entrada e unidade.
- valor fixo de ganho, tabela: 1,3,10,30,100,300,1000,3000
- ganho de usuário: real + digital (algoritmo em HW)
- balanço: auto-zero ou ajuste através de cursor
- calibração por shunt-cal: valor do resistor em ohms, valor dos resistores da ponte -> faixa de entrada e unidade.
- Calibração por regressão linear

ICP

Configuração do Canal

- fios em pinos 8 e 9, GND em pino 3.

Amplificação do Sinal (Ganho), Calibração e ajuste (balanço)

- fundo de escala e sensibilidade -> cálculo de valor de ganho (ganho de usuário) -> faixa de entrada e unidade.
- valor fixo de ganho, tabela: 1,3,10,30,100,300,1000,3000
- ganho de usuário: real + digital (algoritmo em HW)
- balanço: auto-zero ou ajuste através de cursor
- Calibração por regressão linear

Tensão

Configuração do Canal

- selecionar flutuante ou com impedância
- disponibilizar controle de offset
- perguntar se há interesse em alimentar o sensor -> Vexc user

Amplificação do Sinal (Ganho), Calibração e ajuste (offset)

- fundo de escala e sensibilidade -> cálculo de valor de ganho (ganho de usuário) -> faixa de entrada e unidade.
- valor fixo de ganho, tabela: 1,3,10,30,100,300,1000,3000
- ganho de usuário: real + digital (algoritmo em HW)
- balanço: auto-zero ou ajuste através de cursor

- Calibração por regressão linear

Corrente

Configuração do Canal

- selecionar I_IN/E+R
- selecionar II em MUX IN-
- ligar fios em pinos 8 e 1.
- Entrada flutuando

Amplificação do Sinal (Ganho), Calibração e ajuste (balanço)

- fundo de escala e sensibilidade -> cálculo de valor de ganho (ganho de usuário) -> faixa de entrada e unidade.
- valor fixo de ganho, tabela: 1,3,10,30,100,300,1000,3000
- ganho de usuário: real + digital (algoritmo em HW)
- balanço: auto-zero ou ajuste através de cursor
- Calibração por regressão linear

Termopar

Configuração do Canal

- Fios em pinos 1 e 6.
- Define se sensor está externo ou interno

Amplificação do Sinal (Ganho), Calibração e ajuste (balanço)

- fundo de escala e sensibilidade -> cálculo de valor de ganho (ganho de usuário) -> faixa de entrada e unidade.
- valor fixo de ganho, tabela: 1,3,10,30,100,300,1000,3000
- ganho de usuário: real + digital (algoritmo em HW)
- balanço: auto-zero ou ajuste através de cursor
- Calibração por regressão linear

Detalhamento para Configurações Digitais

Gravação

Tipos de gravação:

- contínua
 - Tempo de cada arquivo em segundos
- contínua programada
 - Tempo de cada arquivo em segundos
 - Horário de início
- intermitente (grava durante um tempo, pára, depois continua)
 - tempo de cada arquivo
 - tempo total de espera mais gravação
- intermitente programada (grava durante um tempo, pára, depois continua)
 - tempo de cada arquivo
 - tempo total de espera mais gravação
 - horário de início

Status de gravação:

- inicia gravação

- estado da gravação
- pára gravação

Aquisição

Frequência de amostragem

- entrar com frequência desejada e mostrar frequência real

GPS

- gravar ou não dados e opção de colocar configurar intervalo de tempo para gravar as mensagens (intervalo mínimo de 1s)

Sincronismo

Tipo

- Master
 - GPS
 - Real Time Clock
- Slave
 - RS-485

Relógio

- Get/Set Clock UTC time

IP Config

- Definir configuração de IP e subnet

Detalhamento para Calibração do canal

Cada ganho de cada canal deve ter uma equação linear associada, que será aplicada aos dados brutos. Para calibrar o aparelho:

1. coloque o maior filtro possível para não afetar as medidas
2. aplique o ganho no canal que se deseja calibrar, **com o balanço zerado e desligado**.
3. limpe os dados anteriores de configuração: coloque "0" em Offset_DA, "0" em offset_B e "1" em Gain_M.
4. aplique N+1 tensões na entrada, começando-se por 0 até 10V. Levante a equação da reta $y = A \cdot x + B$, onde y é o valor real medido, x é o valor ideal e B é o valor de offset em níveis do A/D. Lembre-se que A e B são float32.
5. envie para a FW o valor de $m = g/A$, onde g é o ganho atual, junto com o valor de B, que será o offset_B.
6. na FW, faça as seguintes contas:
7. $x = (y - b) \cdot m$
8. $GAIN_M = (int32_t) (m \cdot (1 < 15));$
9. $offset_B = (int32_t) (B \cdot (1 < 15));$
10. $offset_bm = ((int32_t)(1 < 15) \cdot (0,5) - (GAIN_M \cdot offset_B));$
11. para cada amostra:
12. $valor_32 = ((int32_t) valor_medido_16) \cdot GAIN_M$
13. $valor_32 = valor_32 + offset_bm$
14. $valor_final = saturate (valor_32 > 15) ;$

Detalhamento dos circuitos eletrônicos

O circuito eletrônico da placa de canal é baseado e compatível com o projeto ADS 5000 (H6389) onde sua principal característica é a modularidade dos canais.

O circuito da placa mãe é simples e de baixo consumo, além de ser baseado e firmware-compatível com o projeto DLG021 (H6979). Ficou decidido colocar um FPGA para implementar a decimação digital.

Maior detalhamento pode ser obtido nos esquemas elétricos do projeto.

Detalhamento da mecânica

A mecânica do projeto teve como princípio a praticidade de utilizar uma caixa pronta disponível no mercado. A marca BOPLA com o seu perfil ALUBOS foi a que atendeu melhor os requisitos de dureza e praticidade.

O tamanho e o perfil selecionados foram resultados da escolha por 8 canais no sistema, na decisão de ser o menor possível e na exigência de se utilizar o circuito de canal analógico do sistema ADS5000.

Ao mudar o conector placa-placa dos canais analógicos para SMD obteve-se um enorme ganho

Detalhamento da decimação

Existe uma tabela no ARM que define as frequências de amostragem possíveis.

Freq. ACQ.	RATE CIC	SEL CIC	SEL FIR	Freq. Final

Funções básicas

Hardware

Placa Motherboard

- Tensões 5V3, 3V3, 5VREF, 2V5VREF
- Velocidade de comunicação com AVR
- Integridade de sinais de A/D
- Ruído do A/D.
- Sincronismo

Placa Canal

- Ganho 1, 3.
- Seleção Impedância em AC
- Balanço +- 20mV
- Fc em 2048 Hz.
- Seleção de funções do canal (relê)
- Seleção Float/Impedância

- Excitação de pontes com 5V
- Evitar curto na excitação
- Shunt-Cal
- TEDs
- ICP 4mA

Firmware

ARM

- Código de escrita/leitura da flash;
- Aquisição de dados : frequência de aquisição, configuração filtros, receber dados;
- Gravar 8 canais no cartão;
- Código de leitura de junta fria e obtenção de temperatura do termopar;
- Protocolo de comunicação com AVR:
 - Seleção de canal;
 - Seleção de ganho;
 - Seleção do tipo da entrada;
 - Ajuste de D/A offset/balanço;
 - Leitura de junta fria;
 - Ajuste da tensão de excitação;
- Modificações em protocolo UDP
- FPGA
 - Obter .bin via tftp e gravar na flash:
 - Ler .bin da flash e dar boot em fpga:
 - Configuração da freq. de aquisição:
 - Configuração do filtro FIR:
 - Configuração dos sinais de sincronismo:
- Gravar informações do canal analógico em descrição do arquivo TEM;

AVR

- I2C: obtenção de endereço via A/D;
- Rotina para ler/escrever da flash;
- Estados do software: wait CMD, aqusitando temperatura, executando CMD, hibernar;
- SPI: D/A excitação, D/A offset/bal, 74HC595;
- RELÊ: atuar;
- Leitura sensor junta fria;
- SHUNT-CAL: acionar;
- ICP: acionar;
- Imp. de entrada (AC e Float/100K): acionar
- Protocolo:

write:

CMD: W (1 bit MSB – "1") + TAG (7 bits LSB) + CHK + VALUE (WR only, size depends on protocol)

RETURN: TAG + 1.

VALUE MAX SIZE: 26 bytes total.

read:

CMD: R (1 bit MSB – “0”) + TAG (7 bits LSB)

RETURN: TAG + VALUE (size depends on protocol) + CHK (1 byte)

CHECKSUM: APENAS VALUE.

NOME	Permission 1=write 0=read	TAG	Type	Description
TWI_COLD_JUNCTION	Read	0	int16_t	Tensão de junta fria
TWI_CHCONFIG	Write	1	TPoprSetup	Mesma estrutura detalhada abaixo do protocolo UDP – muda as configurações da placa
TWI_SHUNT_CAL	Write	2	int16_t	Shunt-Cal <> 0: connected
TWI_CHGNADJUST	Read / Write	3	Int16_t	Muda valores da calibração do ganho (fora de uso atualmente, apenas grava)
TWI_CHEXCONFIG	Read / Write	4	uint16_t	Muda calibração da tensão de excitação
TWI_CAL_GAIN_OFFSET_DIGITAL	Read	5	uint16_t	Leitura dos ganhos digitais guardados na plaquinha.
TWI_SAVE_ADJUST	Read / Write	6	xxxxx	Impedância de entrada AC 0: desconectada 1: 100 Ω
TWI_LOAD_ADJUST	Read / Write	7	xxxxx	Shunt-Cal <> 0: connected
TWI_READ_VERSION	Read	8	uint16_t	Lê versão da placa
TWI_SET_AMPLIFIER	Read / Write	9	xxxxx	Aplica os ganhos configurados na memória da placa.

- Função de seleção do tipo de canal (atua em MUX, RELÊ, Imp. De Entrada, ICP, 595) ;

FW AVR – função select_sensor

Seleção do Tipo de sensor:

Strain-Gauge

Configuração do Canal:

¼ de ponte

- selecionar E+R em RLA
- selecionar Vexc/2 em MUX IN-
- selecionar entrada tipo flutuante

½ de ponte

- selecionar Vexc
- selecionar Vexc/2 em MUX IN-
- selecionar entrada tipo flutuante

Ponte completa

- selecionar Vexc
- selecionar INP- em entrada negativa
- selecionar entrada tipo flutuante

ICP

Configuração do Canal

- selecionar VEXC/XIN
- selecionar AGND em IN-
- selecionar entrada com impedância DC.
- ligar fonte de corrente

Tensão

Configuração do Canal

- Selecionar VEXC/XIN
- selecionar INPn em IN-
- selecionar com impedância DC

Corrente

Configuração do Canal

- selecionar I_IN/E+R
- selecionar II em MUX IN-
- Entrada flutuando

Termopar

Configuração do Canal

- selecionar VEXC/XIN
- entrada com impedância
- entrada com C+R
- selecionar MUX IN-
- habilitar leitura junta fria interna ou externa (porta TEDs)

FW – função chan_set_default_config()

- desligar fonte de corrente
- colocar DA BAL no meio do fundo de escala
- desligar shunt-cal
- colocar AGND em MUX IN-
- selecionar XIN/VEXC.
- desligar C+R.
- Ganho 1

- Algoritmo de cálculo de ganho (atua em PGA, AMPOP e D/A);
- Modo baixo consumo;
- AVR I2C Bootloader

HDL

- Ler A/D ADS8344
- Implementar decimação (CIC filters)
- Implementar seleção de sincronismo
- Enviar para ARM
- Implementar contadores
- Seleção de quais canais enviar para ARM.
- Implementar SPI para configurar FPGA
- Leitura ADXL

Software

CALIBRADOR:

PROGDLG:

MENSAGENS DO PROTOCOLO

Revisões

1.1

- Novas mensagens
 - pcDLGSetChExAdjust
 - pcDLGGetChExAdjust
- Alterações em mensagens:
 - TpPcDLGAcqSetupR:
 - TpPcDLGSetChGnAdjust:
 - TpPcDLGSetChConfig
 - TpPcDLGSetProgDLogR

Relação das Mensagens

Neste tópico serão descritos os formatos das mensagens do protocolo de comunicação com o coletor de dados DLG0221 ou DLG4K. A porta UDP utilizada pelo DLG para tratar esse protocolo é a port 41401. A tabela abaixo apresenta a lista das mensagens desse protocolo.

PacketCode	Mnemônico	Descrição	Sentido
\$0004	pcDLGSetSecureIP	Programa a configuração do IP do equipamento	PC → DLG
\$0005	pcDLGSetSecureIPR	Resposta ao pcDLGSetSecureIP	DLG → PC
\$0006	pcDLGGetSecureIP	Leitura da configuração do endereço IP do equipamento	PC → DLG
\$0007	pcDLGGetSecureIPR	Resposta ao pcDLGGetSecureIP	DLG → PC
\$0008	pcDLGGetInfo	Solicita informações gerais do aparelho	PC → DLG
\$0009	pcDLGGetInfoR	Resposta ao pcDLGGetInfo	DLG → PC
\$000A	pcDLGSetInfo	Grava informações gerais do aparelho	PC → DLG
\$000B	pcDLGSetInfoR	Resposta ao pcDLGSetInfo	DLG → PC
\$000C	pcDLGGetDateTime	Leitura do relógio de tempo real	PC → DLG
\$000D	pcDLGGetDateTimeR	Resposta ao pcDLGGetDateTime	DLG → PC
\$000E	pcDLGSetDateTime	Ajuste do relógio de tempo real	PC → DLG
\$000F	pcDLGSetDateTimeR	Resposta ao pcDLGSetDateTime	DLG → PC
\$0010	pcDLGGetInfoChan	Solicita informações gerais do aparelho	PC → DLG
\$0011	pcDLGGetInfoChanR	Resposta ao pcDLGGetInfo	DLG → PC
\$0012	pcDLGSetInfoChan	Grava informações gerais do aparelho	PC → DLG
\$0013	pcDLGSetInfoChanR	Resposta ao pcDLGSetInfo	DLG → PC
\$0100	pcDLGGetChGnAdjust	Solicita o ajuste da escala de ganho do canal	PC → DLG
\$0101	pcDLGGetChGnAdjustR	Resposta ao pcDLGGetChAdjust	DLG → PC
\$0102	pcDLGSetChGnAdjust	Programa o ajuste da escala de ganho do canal	PC → DLG

\$0103	pcDLGSetChGnAdjustR	Resposta ao pcDLGSetChAdjust	DLG → PC
\$0106	pcDLGSetChExAdjust	Programa o ajuste da escala de ganho do canal	PC → DLG
\$0107	pcDLGSetChExAdjustR	Resposta ao pcDLGSetChAdjust	DLG → PC
\$0108	pcDLGGetChExAdjust	Solicita o ajuste da tensão de excitação do canal	PC → DLG
\$0109	pcDLGGetChExAdjustR	Resposta ao pcDLGGetChExAdjust	DLG → PC
\$0104	pcDLGSaveAdjust	Salva os ajustes de ganho e offset das escalas de ganho	PC → DLG
\$0105	pcDLGSaveAdjustR	Resposta ao pcDLGSaveAdjust	DLG → PC
\$010E	pcDLGGetAiAdjust	Solicita o ajuste de ganho e offset do canal A/D	PC → DLG
\$010F	pcDLGGetAiAdjustR	Resposta ao pcDLGGetAiAdjust	DLG → PC
\$0110	pcDLGSetAiAdjust	Programa o ajuste de ganho e offset do canal A/D	PC → DLG
\$0111	pcDLGSetAiAdjustR	Resposta ao pcDLGSetAiAdjust	DLG → PC
\$1000	pcDLGAcqSetup	Inicia a aquisição de sinais	PC → DLG
\$1001	pcDLGAcqSetupR	Resposta ao pcDLGAcqSetupR	DLG → PC
\$1002	pcDLGAcqStart	Inicia a aquisição de sinais	PC → DLG
\$1003	pcDLGAcqStartR	Resposta ao pcDLGAcqStart	DLG → PC
\$1006	pcDLGAcqStop	Finaliza a aquisição de sinais	PC → DLG
\$1007	pcDLGAcqStopR	Resposta ao pcDLGAcqStop	DLG → PC
\$100B	pcDLGAcqData	Pacote de dados da aquisição de sinais	PC → DLG
\$2000	pcDLGGetChConfig	Solicita a configuração do canal	PC → DLG
\$2001	pcDLGGetChConfigR	Resposta ao pcDLGGetChConfig	DLG → PC
\$2000	pcDLGGetChConfig	Solicita a configuração do canal	PC → DLG
\$2001	pcDLGGetChConfigR	Resposta ao pcDLGGetChConfig	DLG → PC
\$2002	pcDLGSetChConfig	Programa a configuração do canal	PC → DLG
\$2003	pcDLGSetChConfigR	Resposta ao pcDLGSetChConfig	DLG → PC
\$2004	pcDLGShuntCal	Programa o resistor de shunt cal	PC → DLG
\$2005	pcDLGShuntCalR	Resposta ao pcDLGShuntCal	DLG → PC
\$2006	pcDLGShuntCal	Programa o resistor de shunt cal	PC → DLG
\$2007	pcDLGShuntCalR	Resposta ao pcDLGShuntCal	DLG → PC
\$2008	pcDLGGetUserGain	Solicita o ganho de usuário	PC → DLG
\$2009	pcDLGGetUserGainR	Resposta ao pcDLGGetUserGain	DLG → PC
\$200A	pcDLGSetUserGain	Programa o ganho de usuário	PC → DLG
\$200B	pcDLGSetUserGainR	Resposta ao pcDLGSetUserGain	DLG → PC
\$200C	pcDLGGetUserVexc	Solicita a tensão de excitação de usuário	PC → DLG
\$200D	pcDLGGetUserVexcR	Resposta ao pcDLGGetUserVexc	DLG → PC
\$200E	pcDLGSetUserVexc	Programa a tensão de excitação de usuário	PC → DLG
\$200F	pcDLGSetUserVexcR	Resposta ao pcDLGSetUserVexc	DLG → PC
\$2100	pcDLGSaveSetup	Salva configuração (dos canais) na memória não volátil	PC → DLG
\$2101	pcDLGSaveSetupR	Resposta ao pcDLGSaveSetup	DLG → PC
\$2106	pcDLGResetFlash	Carrega Todas as Configs Default do Sistema e as Salva na memória não volátil	PC → DLG
\$2107	pcDLGResetFlashR	Resposta ao pcDLGResetFlash	DLG → PC
\$3000	pcDLGGetAiSetup	Leitura da configuração dos canais para aquisição	PC → DLG
\$3001	pcDLGGetAiSetupR	Resposta ao pcDLGGetAiSetup	DLG → PC
\$3002	pcDLGSetAiSetup	Escrita da configuração dos canais para aquisição	PC → DLG
\$3003	pcDLGSetAiSetupR	Resposta ao pcDLGSetAiSetup	DLG → PC
\$3004	pcDLGGetProgDLog	Leitura da programação da aquisição de sinais do coletor de dados	PC → DLG
\$3005	pcDLGGetProgDLogR	Resposta ao pcDLGSetAiSetup	DLG → PC
\$3006	pcDLGSetProgDLog	Programação da aquisição de sinais do coletor de dados	PC → DLG
\$3007	pcDLGSetProgDLogR	Resposta ao pcDLGSetProgDLog	DLG → PC
\$3008	pcDLGCommandDLog	Comando para o coletor de dados	PC → DLG
\$3009	pcDLGCommandDLogR	Resposta ao pcDLGCommandDLog	DLG → PC
\$300A	pcDLGGetGPS_Setup	Leitura da configuração da operação do GPS	PC → DLG

\$300B	pcDLGGetGPS_SetupR	Resposta ao pcDLGGetGPS_Setup	DLG → PC
\$300C	pcDLGSetGPS_Setup	Escrita da configuração da operação do GPS	PC → DLG
\$300D	pcDLGSetGPS_SetupR	Resposta ao pcDLGSetGPS_Setup	DLG → PC
\$300C	pcDLGSetGPS_Setup	Escrita da configuração da operação do GPS	PC → DLG
\$300D	pcDLGSetGPS_SetupR	Resposta ao pcDLGSetGPS_Setup	DLG → PC

Estruturas de Dados Comuns

Neste tópico são listadas as estruturas de dados comuns a algumas mensagens do protocolo de comunicação.

CUIDADO: Nos módulos do programa em que as estruturas de dados forem declaradas deve ser utilizada a diretiva do compilador para definir alinhamento byte da estrutura. Isso evita que o compilador inclua bytes *dummies* na estrutura de dados para otimizar o acesso à memória. O mesmo cuidado deve ser tomado na declaração das estruturas das mensagens. Do contrário as mensagens enviadas ficariam erradas.

CUIDADO: Esta especificação assume que a ordem de armazenamento dos bytes na memória é o *little endian*. Essa é a ordem utilizada pela Intel nos seus processadores para PC.

Estrutura TpDateTime

Esta estrutura de dados é utilizada para representação da data e hora.

```
//----- Formato da data / hora --
typedef struct _TpDLGDateTime {
    uint16_t      Day;      // Dia
    uint16_t      Month;    // Mês
    uint16_t      Year;     // Ano
    uint16_t      Hour;     // Hora
    uint16_t      Minute;   // Minuto
    uint16_t      Second;   // Segundo
    uint16_t      MS;       // ms
} TpDLGDateTime, *PTpDLGDateTime;
```

Estrutura TpPcDLGGeneric

Esta estrutura de dados é utilizada na mensagem de requisição genérica.

```
//----- Tipo para mensagem de requisição genérica --
typedef struct _TpPcDLGGeneric {
    uint16_t      PacketCode; // Código do pacote
    uint16_t      Reserved;   // reservado
} TpPcDLGGeneric, *PTpPcDLGGeneric;
```

Estrutura TpPcDLGResp

Esta estrutura de dados é utilizada na mensagem de resposta genérica.

```
//----- Tipo para mensagem de resposta genérica --
typedef struct _TpPcDLGGenericResp {
    uint16_t      PacketCode; // Código do pacote
    uint16_t      Reserved;   // reservado
}
```

```

uint16_t      Error;          // <> 0 indica erro
} TpPcDLGGenericResp, *PTpPcDLGGenericResp;

```

MENSAGENS BÁSICAS E DE IDENTIFICAÇÃO

Neste tópico serão descritos os formatos das mensagens básicas e de identificação do aparelho.

Essas mensagens são utilizadas para a configuração do endereço IP do aparelho, leitura e gravação das informações de fábrica do aparelho (tais como: número de série, opções de ganho, filtro e tensão de excitação), leitura e ajuste do relógio de tempo real (RTC) e leitura do status de erro do equipamento.

Configuração do Endereço IP (pcDLGSetSecureIP)

Propósito:	Esta mensagem é utilizada na programação do endereço IP do equipamento.
Formato da Mensagem:	TpPcDLGSetSecureIP
Packet Code:	pcDLGSetSecureIP
Formato da Resposta:	TpPcDLGGenericResp
Packet Code:	pcDLGSetSecureIPR
Observações:	O campo <i>DisBlockIP</i> quando igual a zero restringe a resposta do equipamento apenas ao computador com o endereço IP especificado no campo <i>OwnerIP</i>. O campo <i>sTag</i> é um string com tamanho máximo de 10 caracteres para a identificação do equipamento.

```

//----- Tipo para as mensagens pcDLGSetSecureIP e pcDLGGetSecureIPR --
typedef struct _TpPcDLGSetSecureIP {
    uint16_t      PacketCode;    // Código do pacote
    uint16_t      Reserved;      // reservado
    uint16_t      Error;         // <> 0 indica erro
    uint8_t       DeviceIP[4];   // endereço IP do equipamento
    uint16_t      DisBlockIP;    // <> 0: inibe a restrição de resposta
                                // para o OwnerIP
    uint8_t       OwnerIP[4];    // endereço IP para o qual o equipamento
                                // pode responder
    uint8_t       sTag[10];      // Tag do equipamento
} TpPcDLGSetSecureIP, *PTpPcDLGSetSecureIP,
TpPcDLGGetSecureIPR, *PTpPcDLGGetSecureIPR;

```

Leitura da Configuração de Endereço IP (pcDLGGetSecureIP)

Propósito:	Esta mensagem é utilizada para a leitura da configuração do endereço IP do equipamento.
Formato da Mensagem:	TpPcDLGGeneric
Packet Code:	pcDLGGetSecureIP
Formato da Resposta:	TpPcDLGSetSecureIPR
Packet Code:	pcDLGGetSecureIPR
Observações:	

Gravação das Informações de Fábrica do Equipamento (pcDLGSetInfo)

Propósito:	Esta mensagem é reservada para uso interno da Lynx e é utilizada para gravar as informações gerais do equipamento, como por exemplo, modelo, número de série e opções de filtro e tensão de excitação.
Formato da Mensagem:	TpPcDLGSetInfo
Packet Code:	pcDLGSetInfo
Formato da Resposta:	TpPcDLGGenericResp
Packet Code:	pcDLGSetInfoR

Observações:

Os campos DisBlockIP, OwnerIP e sTag são os mesmos das mensagens pcDLGSetSecureIP e pcDLGGetSecureIPR.

O campo sModel é uma string terminada em zero com até 16 caracteres utilizados para a identificação do modelo do equipamento.

A versão do hardware e do firmware do equipamento são codificados nos campos VersionP1 e VersionP2 conforme descritos nos comentários da estrutura de dados.

O tipo TpRealLynx é um inteiro de 16 bits sem sinal utilizado para representar um número real em formato compactado. Esse formato é utilizado para representar as opções de frequência de corte dos filtros, opções de ganho e de tensão de excitação. Os três nibbles menos significativos definem a mantissa e o nibble mais significativo a potência de 10. O bit mais significativo do nibble da potência indica o sinal da potência. Exemplos: o valor compactado 0x9005 representa o valor real 0.5 e o valor compactado 0x3002 representa o valor real 2000.

O campo CondGain é uma lista com as opções de ganho dos condicionadores de sinais do DLG. Os valores nominais dos ganhos são representados no formato TpRealLynx. A lista pode ter até 16 opções de ganho. Caso o número de opções de ganho seja menor que 16, após a última opção deve ser armazenado o valor binário 65535.

Analogamente os campos ListLPF e SensPwr apresentam respectivamente as opções de frequência de corte do filtro passa baixas e as opções de tensão de excitação dos sensores.

O DLG possui 5 tensões de excitação : 1V, 2,5V, 3,3V, 5V, user e dois filtros digitais passa-baixas: 1/10 e 1/3 de Fa/2.

O DLG não possui filtros passa altas, com isso o campo ListHPF deve ser preenchido com o valor binário 65535.

O DLG também não possui suporte para filtros notch, com isso o campo Notch deve ser igual a zero.

Os campos sTechnician, sAdjustDat e dtAdjust2K devem ser ignorados pelo DLG. Eles são passados para o DLG na mensagem pcDLGSaveAdjust.

```
#define TpRealLynx uint16_t;

//----- Tipo para as mensagens pcDLGSetInfo e pcDLGGetInfoR --
typedef struct _TpPcDLGSetInfo {
    uint16_t      PacketCode;    // Código do pacote
    uint16_t      Reserved;      // reservado
    uint16_t      Error;         // <> 0 indica erro
    uint16_t      DisBlockIP;    // <> 0: inibe a restrição de resposta
                                // para o OwnerIP
    uint8_t       OwnerIP[4];    // endereço IP para o qual o equipamento
                                // pode responder
    uint8_t       sTag[10];      // Tag do equipamento
    uint8_t       sModel[16];    // Modelo do equipamento
    uint32_t      SerialNo;      // Número de série do equipamento
    uint32_t      NIP1;          // NIP da parte 1 (placa principal)
    uint32_t      FPGA_Version;  // FPGA Code Version
    uint32_t      NIP3;          // NIP da parte 3 (N.A.)
    uint16_t      VersionP1;     // Versão da placa principal
                                // Versão do Hw (LSB: v.r) e Sw (MSB: v.r.b) da
    Parte 1
```

```

// v e r são nibbles. O built do firmware é
// obtido do LSB do Version P3
// (MSB ignorado pelo DLG no pcDLGSetInfo)
uint16_t      VersionP2;    // Versão da parte 2 (N.A.)
uint16_t      VersionP3;    // Informa a montagem (build) do firmware
(LSB)

// (ignorado pelo DLG no pcDLGSetInfo)
TpRealLynx    CondGain[16]; // Ganho dos condicionadores (ignorado)
// usa formato real compactado (Lynx)
uint16_t      ListF_ACQ[16]; // Frequências disponíveis para
gravação ( 65535 é o fim da tabela)
TpRealLynx    ListLPF[16]; // Frequência de corte do filtro passa baixas
(ignorado)
uint16_t      Chan_Opt;     // Número de canais existentes e o que
está disponível de acessório em
// FW e HW
// bits 0-7 : número de canais analógicos
presentes
// bit 8: GPS (1=TRUE)
// bit 9: Rainflow
// bit 10: Estatística
// bit 11:
TpRealLynx    SensPwr[16]; // Tensões de excitação disponíveis
char_t        sTechnician[16]; // Técnico responsável pela
calibração
// (ignorado pelo DLG no pcDLGSetInfo)
char_t        sAdjustDat[11]; // Data da calibração
// (ignorado pelo DLG no pcDLGSetInfo)
uint32_t      dtAdjust2K;    // Idem no formato: número de segundos
deste
// 2000/01/01 00:00:00 (uint32)
// (ignorado pelo DLG no pcDLGSetInfo)
} TpPcDLGSetInfo, *PTpPcDLGSeInfo, TpPcDLGGetInfoR, *PTpPcDLGGeInfoR;

```

Leitura das Informações de Fábrica do Equipamento (pcDLGGetInfo)

Propósito:	Esta mensagem é utilizada pela aplicação do microcomputador para a leitura dos dados gerais de identificação do equipamento.
Formato da Mensagem:	TpPcDLGGeneric
Packet Code:	pcDLGGetInfo
Formato da Resposta:	TpPcDLGGetInfoR
Packet Code:	pcDLGGetInfoR
Observações:	Ao contrário da mensagem pcDLGSetInfo, os campos sTechnician, sAdjustDat e dtAdjust2K são utilizados na mensagem de resposta à mensagem pcDLGGetInfo.

Gravação das Informações de Fábrica do Equipamento (pcDLGSetInfoChan)

Propósito:	Esta mensagem é reservada para uso interno da Lynx e é utilizada para gravar as informações de cada placa de canal analógico, como por exemplo, modelo, número de série e opções de filtro e tensão de excitação.
Formato da Mensagem:	TpPcDLGSetInfoChan
Packet Code:	pcDLGSetInfoChan
Formato da Resposta:	TpPcDLGGenericResp
Packet Code:	pcDLGSetInfoChanR
Observações:	O campo sModel é uma string terminada em zero com até 16 caracteres utilizados para a identificação do modelo do canal, e é um valor pré-configurado na FW. A versão do hardware e do firmware do canal são codificados nos campos VersionP1 e VersionP2 conforme descritos nos comentários da estrutura de dados. O campo CondGain é uma lista com as opções de ganho dos condicionadores de sinais do DLG. Os valores nominais dos ganhos são representados no formato inteiro de 16 bits, DIFERENTE do usado em SetInfo. A lista pode ter até 16 opções de ganho. Caso o número de opções de ganho seja menor que 16, após a última opção deve ser armazenado o valor binário 65535. Analogamente o campo SensPwr apresenta as opções de tensão de excitação dos sensores, no formato TpRealLynx O DLG também não possui suporte para filtros notch, com isso o campo Notch deve ser igual a zero. Os campos sTechnician e dtAdjust2K devem ser ignorados pelo DLG. Eles são passados para o DLG na mensagem pcDLGSaveAdjust.

```

#define TpRealLynx uint16_t;
//----- Tipo para as mensagens pcDLGSetInfo e pcDLGGetInfoR --
typedef struct _TpPcDLGSetInfoChan {
    uint16_t      PacketCode;  // Código do pacote
    uint16_t      Reserved;    // reservado
    uint16_t      Error;       // <> 0 indica erro
    uint16_t      Channel;     // Canal (0-7)
    uint8_t       sModel[12];  // modelo da plaquinha
    uint32_t      NIP1;        // grava NIP do canal
    uint32_t      dtAdjust2K;  // Data da calibração no formato: número
de segundos deste           // 2000/01/01 00:00:00 (uint32)
                                Error_Ch;    // <> 0 indica erro na plaquinha
    uint16_t      VersionP1;   // Versão da placa principal
                                // Versão do Sw (MSB: v.r.b) da Parte 1
                                // v e r são nibbles. O built do firmware é
                                // obtido do LSB do Version P3
                                // (LSB ignorado pelo DLG no pcDLGSetInfoChan)
    uint16_t      VersionP2;   // Versão da parte 2 (N.A.) (ignorado em
pcDLGSetInfoChan)

```

```

uint16_t          VersionP3;    // Informa a montagem (built) do firmware
(LSB)
                                // (ignorado pelo DLG no pcDLGSetInfoChan)
uint16_t          CondGain[16]; // ganhos da plaquinha em uint16_t
( valor em inteiro decimal, 0 a 3000 )
                                //
TpRealLynx    SensPwr[16]; // tensões da plaquinha (É TpRealLynx)
                                //
uint32_t          Reserved2;
} TpPcDLGSetInfoChan, *PTpPcDLGSeInfoChan, TpPcDLGGetInfoChanR,
*PTpPcDLGGeInfoChanR;

```

Leitura das Informações de Fábrica do Equipamento (pcDLGGetInfoChan)

Propósito:	Esta mensagem é utilizada pela aplicação do microcomputador para a leitura dos dados gerais de identificação do equipamento.
Formato da Mensagem:	TpPcDLGGetInfoChan
Packet Code:	pcDLGGetInfoChan
Formato da Resposta:	TpPcDLGGetInfoChanR
Packet Code:	pcDLGGetInfoChanR
Observações:	Ler o número de canais disponíveis em Setinfo e buscar apenas estes, um a um.

```

//----- Tipo para as mensagens pcDLGGetInfoChan e pcDLGGetInfoChan --
typedef struct _TpPcDLGGetInfoChan {
    uint16_t          PacketCode;    // Código do pacote
    uint16_t          Reserved;      // reservado
    uint16_t          Error;          // <> 0 indica erro
    uint16_t          Channel;        // Número do canal
} TpPcDLGGetInfoChan, *PTpPcDLGGetInfoChan;

```

Ajuste do Relógio de Tempo Real (pcDLGSetDateTime)

Propósito:	Esta mensagem é utilizada pela aplicação do microcomputador para a leitura dos dados gerais de identificação do equipamento.
Formato da Mensagem:	TpPcDLGGeneric
Packet Code:	pcDLGGetInfo
Formato da Resposta:	TpPcDLGGetInfoR
Packet Code:	pcDLGGetInfoR
Observações:	Ao contrário da mensagem pcDLGSetInfo, os campos sTechnician,

sAdjustDat e dtAdjust2K são utilizados na mensagem de resposta à mensagem pcDLGGetInfo.

```
//----- Tipo para a mensagem pcDLGGetDateTime --  
typedef struct _TpPcDLGSetDateTime {  
    uint16_t      PacketCode;    // Código do pacote  
    uint16_t      Reserved;      // reservado  
    uint16_t      Error;          // <> 0 indica erro  
    TpDLGDateTime DateTime;  
} TpPcDLGGetDateTime, *PTpPcDLGGetDateTime;
```

Leitura do Relógio de Tempo Real (pcDLGGetDateTime)

Propósito:	Esta mensagem é utilizada pela aplicação do microcomputador para a leitura dos dados gerais de identificação do equipamento.
Formato da Mensagem:	TpPcDLGSetDateTime
Packet Code:	pcDLGSetDateTime
Formato da Resposta:	TpPcDLGGenericResp
Packet Code:	pcDLGSetDateTimeR
Observações:	

```
//----- Tipo para a mensagem pcDLGGetDateTimeR --  
typedef struct _TpPcDLGGetDateTimeR {  
    uint16_t      PacketCode;    // Código do pacote  
    uint16_t      Reserved;      // reservado  
    uint16_t      Error;          // <> 0 indica erro  
    TpDLGDateTime DateTime;  
} TpPcDLGGetDateTimeR, *PTpPcDLGGetDateTimeR;
```

MENSAGENS PARA AJUSTE DE FÁBRICA

Neste tópico serão descritos os formatos das mensagens utilizadas internamente pela Lynx para a calibração das escalas de ganho dos condicionadores de sinais.

Programa o Ajuste de Escala de Ganho de um Canal (pcDLGSetChGnAdjust)

Propósito:	Esta mensagem é utilizada para passar o ajuste de uma escala de ganho de um canal. A escala de ganho deve ser ajustada de modo que o ganho entre a saída do canal (em +/- 10V) e a entrada do condicionador tenha o menor erro em relação ao ganho nominal.
Formato da Mensagem:	TpPcDLGSetChGnAdjust
Packet Code:	pcDLGSetChGnAdjust
Formato da Resposta:	TpPcDLGGenericResp
Packet Code:	pcDLGSetChGnAdjustR
Observações:	O campo Channel corresponde ao canal a ser ajustado (0 a 8). O campo iGain corresponde ao índice da escala de ganho a ser ajustada (0 a nGain-1). O valor de nGain corresponde ao número de escalas de ganhos do equipamento. O padrão de fábrica deve ter os seguintes ganhos: 1, 3,10,30,100,300,1000,3000. O ajuste da escala de ganho é determinado pelos campos Offset_DA, Offset_B, Gain_M. O ajuste analógico do offset é dado pelo campo Offset_DA (-32768 a 32767). Esse controle é aplicado no circuito de entrada. O ajuste digital do offset é dado pelo campo Offset_B (-32768,0 a 32767,0). O campo Gain_M corresponde ao ganho digital necessário para se obter o ganho pretendido.

```
//----- Tipo para as mensagens pcDLGSetChGnAdjust e pcDLGGetChGnAdjustR --
typedef struct _TpPcDLGSetChGnAdjust {
    uint16_t      PacketCode;    // Código do pacote
    uint16_t      Reserved;      // reservado
    uint16_t      Error;         // <> 0 indica erro
    uint16_t      Channel;       // Número do canal
    uint16_t      iGain;         // Índice do ganho (0..nGain-1)
    int16_t       Offset_DA;     // Ajuste de offset do primeiro estágio -
D/A de BAL (fino)
    float32_t     Offset_B;      // Ajuste de offset digital
    float32_t     Gain_M;        // Ganho digital
} TpPcDLGSetChGnAdjust, *PTpPcDLGSetChGnAdjust,
TpPcDLGGetChGnAdjustR, *PTpPcDLGGetChGnAdjustR;
```

Solicita o Ajuste de Escala de Ganho de um Canal (pcDLGGetChGnAdjust)

Propósito:	Esta mensagem é utilizada pelo programa de calibração para carregar o ajuste de uma escala de ganho de um canal.
Formato da Mensagem:	TpPcDLGGetChGnAdjust
Packet Code:	pcDLGGetChGnAdjust
Formato da Resposta:	TpPcDLGGetChGnAdjustR

Packet Code:	pcDLGGetChGnAdjustR
---------------------	---------------------

Observações:	
---------------------	--

```
//----- Tipo para as mensagens pcDLGGetChGnAdjust e pcDLGGetChGnAdjustR --
typedef struct _TpPcDLGSetChGnAdjust {
    uint16_t      PacketCode;  // Código do pacote
    uint16_t      Reserved;    // reservado
    uint16_t      Error;       // <> 0 indica erro
    uint16_t      Channel;     // Número do canal
    uint16_t      iGain;       // Índice do ganho (0..nGain-1)
} TpPcDLGGetChGnAdjust, *PTpPcDLGGetChGnAdjust,
```

Programa o Ajuste da Tensão de Excitação de um Canal (pcDLGSetChExAdjust)

Propósito:	Esta mensagem é utilizada para passar o ajuste de uma escala de excitação de um canal. A escala de ganho deve ser ajustada de modo que o ganho entre a saída do canal (em +/- 10V) e a entrada do condicionador tenha o menor erro em relação ao ganho nominal.
-------------------	--

Formato da Mensagem:	TpPcDLGSetChExAdjust
-----------------------------	----------------------

Packet Code:	pcDLGSetChExAdjust
---------------------	--------------------

Formato da Resposta:	TpPcDLGGenericResp
-----------------------------	--------------------

Packet Code:	pcDLGSetChExAdjustR
---------------------	---------------------

Observações:	O campo Channel corresponde ao canal a ser ajustado (0 a 8). O campo iSensPwr corresponde ao índice da escala de ganho a ser ajustada (0 a nSensPwr-1). O valor de nSensPwr corresponde ao número de escalas de excitação do equipamento. O padrão de fábrica deve ter os seguintes valores: "1", "2,5", "3,3", "5V". O ajuste da escala de ganho é determinado pelo campo SensPwr_DA.
---------------------	--

```
//----- Tipo para as mensagens pcDLGSetChExAdjust --
typedef struct _TpPcDLGSetChExAdjust {
    uint16_t      PacketCode;  // Código do pacote
    uint16_t      Reserved;    // reservado
    uint16_t      Error;       // <> 0 indica erro
    uint16_t      Channel;     // Número do canal
    uint16_t      iSensPwr;    // Índice da tensão de excitação
    (0..nGain-1)
    int16_t      SensPwr_DA;   // -32768 a 32767
} TpPcDLGSetChExAdjust, *PTpPcDLGSetChExAdjust,
TpPcDLGGetChExAdjustR, *PTpPcDLGGetChExAdjustR;
```

Solicita o Ajuste de Escala de Excitação de um Canal (pcDLGGetChExAdjust)

Propósito:	Esta mensagem é utilizada pelo programa de calibração para carregar o ajuste de uma escala de ganho de um canal.
Formato da Mensagem:	TpPcDLGGetChExAdjust
Packet Code:	pcDLGGetChExAdjust
Formato da Resposta:	TpPcDLGGetChExAdjustR
Packet Code:	pcDLGGetChExAdjustR
Observações:	

```
//----- Tipo para as mensagens pcDLGGetChExAdjust --
typedef struct _TpPcDLGGetChExAdjust {
    uint16_t      PacketCode;    // Código do pacote
    uint16_t      Reserved;      // reservado
    uint16_t      Channel;       // Número do canal
    uint16_t      iSensPwr;      // Índice da tensão de excitação
    (0..nGain-1)
} TpPcDLGGetChExAdjust, *PTpPcDLGGetChExAdjust;
```

Salva os Ajustes de Ganho e Offset das Escalas de Ganho (pcDLGSaveAdjust)

Propósito:	Esta mensagem é utilizada para salvar os ajustes de todas as escalas de ganho de todos os canais.
Formato da Mensagem:	TpPcDLGGeneric
Packet Code:	pcDLGSaveAdjust
Formato da Resposta:	TpPcDLGGenericResp
Packet Code:	pcDLGSaveAdjustR
Observações:	O campo sTechnician é uma string terminada com 0 onde deve ser especificado o nome do técnico que efetuou a calibração das escalas de ganho. Os campos sAdjustDat e dtAdjust2K correspondem à data e hora da calibração. Os ajustes são salvos em memória não volátil. Caso a gravação tenha sido efetuado com sucesso, o campo Error da mensagem resposta deve retornar com valor 0.

```
//----- Tipo para a mensagem pcDLGSaveAdjust --
typedef struct _TpPcDLGSaveAdjust {
    uint16_t      PacketCode;    // Código do pacote
    uint16_t      Reserved;      // reservado
    uint16_t      Channel;
    uint16_t      Reserved2;
    char_t        sTechnician[16]; // Técnico responsável pela
    calibração
}
```

```

        char_t          sAdjustDat[12];      // Data da calibração
        uint32_t        dtAdjust2K;        // Idem no formato: número de segundos
deste
                                // 2000/01/01 00:00:00 (uint32)
} TpPcDLGSaveAdjust, *PTpPcDLGSaveAdjust;

```

MENSAGENS PARA AQUISIÇÃO DE SINAIS EM TEMPO REAL

Mensagem de Configuração da Aquisição (pcDLGGetAcqSetup)

Propósito:	A aplicação do microcomputador utiliza esta mensagem para configurar a aquisição de sinais em tempo real.
Formato da Mensagem:	TpPcDLGAcqSetup
Packet Code:	pcDLGAcqSetup
Formato da Resposta:	TpPcDLGAcqSetupR
Packet Code:	pcDLGAcqSetupR
Observações:	Os campos <i>ClockDiv</i>, <i>Period</i> e <i>PreScSw</i> são ignorados atualmente no DLG 4K, o firmware fará todos os cálculos automaticamente e retornará com uma mensagem com o valor obtido. O campo <i>fUseAiAdjust</i> também não está sendo usado. O número de canais de entrada analógicas a serem aquisitados é passado no campo <i>nSignals</i>. Os canais a serem aquisitados são especificados no vetor <i>ICM</i>. Os canais devem ser aquisitados na ordem informada em <i>ICM</i>. O campo <i>nBurstsPack</i> define o número de amostras por canal que devem ser transmitidos em cada pacote de dados aquisitados (veja mensagem pcDLGAcqData).

```

#define                N_MAX_CHAN                8
//----- Tipo para a mensagem pcDLGAcqSetup --
typedef struct _TpPcDLGAcqSetup {
    uint16_t            PacketCode;    // Código do pacote
    uint16_t            Reserved;      // reservado
    float32_t           SampleFreq;    // Frequência de amostragem em Hz
    uint16_t            ClockDiv;      // Clock divider do SSC clock mode
register (0..4095) - Ignorado
    uint16_t            Period;        // Receive clock divider do SSC
clock mode register - Ignorado
                                // (0..255)
    uint16_t            PreScSw;       // Prescaler de software - Ignorado
    uint16_t            fUseAiAdjust;  // <> 0: deve efetuar o ajuste do
A/D por software

```

```

        int16_t          nBurstsPack; // Número de amostragens por pacote por
canal
        int16_t          nSignals;    // Número de canais habilitados
        int16_t          ICM[N_MAX_CHAN]; // Memória de canais a serem
aquistados
    } TpPcDLGAcqSetup, *PTpPcDLGAcqSetup;

//----- Tipo para a mensagem pcDLGAcqSetup --
typedef struct _TpPcDLGAcqSetupR {
    uint16_t          PacketCode; // Código do pacote
    uint16_t          Reserved;   // reservado
    uint16_t          Error;
    float32_t         SampleFreq; // Frequência de amostragem Real do
aparelho em Hz
} TpPcDLGAcqSetup, *PTpPcDLGAcqSetup;

```

Mensagem de Início de Aquisição (pcDLGAcqStart)

Propósito:	A aplicação do microcomputador envia essa mensagem para o coletor de dados iniciar a aquisição de sinais em tempo real.
Formato da Mensagem:	TpPcDLGGeneric
Packet Code:	pcDLGAcqStart
Formato da Resposta:	TpPcDLGGenericResp
Packet Code:	pcDLGAcqStartR
Observações:	Se o DLG estiver operando no modo coletor de dados e a frequência de amostragem definida no modo coletor de dados e o passado na mensagem pcAcqSetup forem diferentes, o comando de início de aquisição de sinais deve ser rejeitado (código de erro diferente de zero).

Mensagem de Finalização da Aquisição (pcDLGAcqStop)

Propósito:	A aplicação do microcomputador envia essa mensagem para o coletor de dados finalizar a aquisição de sinais em tempo real.
Formato da Mensagem:	TpPcDLGGeneric
Packet Code:	pcDLGAcqStop
Formato da Resposta:	TpPcDLGGenericResp
Packet Code:	pcDLGAcqStopR
Observações:	

Pacote de Dados da Aquisição (pcDLGAcqData)

Propósito:	O pacote de dados da aquisição é enviado constantemente pelo coletor de dados após a habilitação da aquisição de sinais em tempo real.
Formato da Mensagem:	TpPcDLGAcqSetup
Packet Code:	pcDLGAcqData
Observações:	O campo FrameNo é zerado pelo coletor de dados no início da aquisição de sinais aquisição e deve ser automaticamente incrementado a cada pacote "aquisitado". Note que esse campo contabiliza o número de pacotes adquiridos e não o número de pacotes enviados. Os campos SampleFreq, nSignals e nBurstsPack devem ser iguais aos respectivos valores passados pela aplicação do microcomputador na mensagem pcDLGAcqSetup. O número total de amostras passadas, no campo Samples, em cada pacote pcDLGAcqData é igual a nSignals vezes nBurstsPack. As amostras são dispostas no vetor Samples na ordem em que elas são adquiridas. No começo de vetor Samples são armazenadas as amostras correspondentes ao primeiro burst de leitura dos canais A/D habilitados. Segue-se a essas amostras o segundo burst e assim sucessivamente até completar o número de bursts especificados em nBurstsPack.

```
//----- Tipo para a mensagem pcDLGAcqData --
typedef struct _TpPcDLGAcqData {
    uint16_t      PacketCode;    // Código do pacote
    uint16_t      Reserved;      // reservado
    int32_t       FrameNo;       // Número do pacotes de dados
    int32_t       TimeStamp;     // Time stamp em ms
    float32_t     SampleFreq;    // Frequência de amostragem em Hz
    int16_t       nBurstsPack;   // Número de amostragens por pacote por
canal
    int16_t       nSignals;      // Número de canais habilitados
    int32_t       ReservedL2;    // reservado
    int32_t       ReservedL3;    // reservado
    int32_t       ReservedL4;    // reservado
    int16_t       Samples[720];  // Amostras adquiridas analógicas
} TpPcDLGAcqData, *PTpPcDLGAcqData;
```

MENSAGENS PARA A CONFIGURAÇÃO DOS CONDICIONADORES

Neste tópico serão descritos os formatos das mensagens utilizadas na configuração dos condicionadores de sinais.

Configura o Condicionador de Sinais (pcDLGSetChConfig)

Propósito:	Esta mensagem é utilizada pela aplicação do microcomputador para configurar o canal do condicionador de sinais.
-------------------	---

Formato da Mensagem:	TpPcDLGSetChConfig
Packet Code:	pcDLGSetChConfig
Formato da Resposta:	TpPcDLGGenericResp
Packet Code:	pcDLGSetChConfigR
Observações:	Veja os comentários referentes aos campos da estrutura de dados tSensor: 0. tensão 1. corrente 2. ponte completa 3. ¼ de ponte 4. ½ ponte 5. ICP 6. termopar E 7. termopar J 8. termopar K 9. termopar T 10. termopar S 11. termopar B 12. termopar N 13. termopar R 14. PT100 15. tensão + offset 16. tensão + GND

```

//----- Tipo para as mensagens pcDLGSetChConfig e pcDLGGetChConfigR --
typedef struct _TpPcDLGSetChConfig {
    uint16_t      PacketCode;    // Código do pacote
    uint16_t      Reserved;      // reservado
    uint16_t      Error;         // <> 0 indica erro
    int16_t       Channel;       // Número do canal
    int16_t       tSensor;       // Índice do tipo da entrada
    (0...N_TIPOS_AI - 1)
    int16_t       iGain;         // Índice do ganho (0..nGain-1)

    int16_t       iLPF_Fc;       // Índice da frequência de corte do filtro
    LPF           // (0..nLPF_Fc-1 )
    int16_t       iSensPwr;      // Índice da tensão de excitação
    (0..nSensPwr-1)
    int16_t       Balance;       // Ajuste de balanço
    uint16_t      fInputDCImp    // Impedância de entrada: 1: 100K,  0:
    Float         fInputACImp    // Impedância de entrada AC <> 0:
    conectada.
    uint16_t      fUseBal;       // <> 0: usa o balanço
} TpPcDLGSetChConfig, *PTpPcDLGSetChConfig,
TpPcDLGGetChConfigR, *PTpPcDLGGetChConfigR;

```

Solicita a Configuração do Condicionador (pcDLGGetChConfig)

Propósito:	Esta mensagem é utilizada pelo programa de calibração para a carregar o ajuste de ganho e offset de um canal A/D.
-------------------	---

Formato da Mensagem:	TpPcDLGGeneric
Packet Code:	pcDLGGetChConfig
Formato da Resposta:	TpPcDLGGetChConfig
Packet Code:	pcDLGGetChConfig
Observações:	

Programa o Ganho do Usuário (pcDLGSetUserGain)

Propósito:	Esta mensagem é utilizada pela aplicação do microcomputador para configurar a escala de ganho do usuário para um canal do condicionador de sinais.
Formato da Mensagem:	TpPcDLGSetUserGain
Packet Code:	pcDLGSetUserGain
Formato da Resposta:	TpPcDLGGenericResp
Packet Code:	pcDLGSetUserGainR
Observações:	Os campos da mensagem são análogos aos da mensagem pcDLGSetChAdjust utilizada para a configuração das escalas de ganho de fábrica. O ganho nominal passado no campo <i>GainNom</i> é apenas informativo e é utilizado pela aplicação do microcomputador para informar ao usuário as posições das chaves de seleção de ganho e/ou atenuação determinados manualmente pelo dip switch da entrada do canal do condicionador.

```
//----- Tipo para as mensagens pcDLGSetUserGain e pcDLGGetUserGainR --
typedef struct _TpPcDLGSetUserGain {
    uint16_t      PacketCode; // Código do pacote
    uint16_t      Reserved; // reservado
    uint16_t      Error; // <> 0 indica erro
    int16_t       Channel; // Número do canal
    int16_t       iGain; // Índice do ganho (nGain-1)
    float32_t     GainNom; // Ganho nominal
} TpPcDLGSetUserGain, *PTpPcDLGSetUserGain,
TpPcDLGGetUserGainR, *PTpPcDLGGetUserGainR;
```

Solicita a Configuração do Ganho do Usuário (pcDLGGetUserGain)

Propósito:	Esta mensagem é utilizada pelo programa de calibração para a carregar o ajuste de uma escala de ganho de um canal.
-------------------	--

Formato da Mensagem:	TpPcDLGGeneric
Packet Code:	pcDLGGetUserGain
Formato da Resposta:	TpPcDLGGetUserGainR
Packet Code:	pcDLGSetUserGainR
Observações:	

Programa a Tensão de Excitação do Usuário (pcDLGSetUserVexc)

Propósito:	Esta mensagem é utilizada pela aplicação do microcomputador para configurar a escala de ganho do usuário para um canal do condicionador de sinais.
Formato da Mensagem:	TpPcDLGSetUserVexc
Packet Code:	pcDLGSetUserVexc
Formato da Resposta:	TpPcDLGGenericResp
Packet Code:	pcDLGSetUserVexcR
Observações:	Os campos da mensagem são análogos aos da mensagem pcDLGSetChAdjust utilizada para a configuração das escalas de ganho de fábrica. A tensão nominal passado no campo <i>GainNom</i> é apenas informativo e é utilizado pela aplicação do microcomputador para informar ao usuário as posições das chaves de seleção de ganho e/ou atenuação determinados manualmente pelo dip switch da entrada do canal do condicionador.

```
//----- Tipo para as mensagens pcDLGSetUserVexc e pcDLGGetUserVexcR --
typedef struct _TpPcDLGSetUserVexc {
    uint16_t      PacketCode; // Código do pacote
    uint16_t      Reserved; // reservado
    uint16_t      Error; // <> 0 indica erro
    int16_t       Channel; // Número do canal
    int16_t       iVexc; // Índice da excitação (nVexc-1)
    float32_t     VexcNom; // Tensão nominal
} TpPcDLGSetUserVexc, *PTpPcDLGSetUserVexc,
TpPcDLGGetUserVexcR, *PTpPcDLGGetUserVexcR;
```

Solicita a Configuração da Tensão de Excitação do Usuário (pcDLGGetUserVexc)

Propósito:	Esta mensagem é utilizada pelo programa de calibração para a carregar o ajuste de uma escala de ganho de um canal.
-------------------	--

Formato da Mensagem:	TpPcDLGGeneric
Packet Code:	pcDLGGetUserVexc
Formato da Resposta:	TpPcDLGGetUserVexcR
Packet Code:	pcDLGSetUserVexcR
Observações:	

Programa o Resistor de Shunt CAL (pcDLGShuntCal)

Propósito:	Esta mensagem é utilizada pela aplicação do microcomputador para selecionar e conectar ou desconectar o resistor de calibração.
Formato da Mensagem:	TpPcDLGShuntCal
Packet Code:	pcDLGShuntCal
Formato da Resposta:	TpPcDLGGenericResp
Packet Code:	pcDLGShuntCalR
Observações:	Veja os comentários referentes aos campos da estrutura de dados TpPCDLGShuntCal.

```
//----- Tipo para a mensagem pcDLGShuntCal -----
typedef struct _TpPcDLGShuntCal {
    uint16_t      PacketCode;    // Código do pacote
    uint16_t      Reserved;      // reservado
    int16_t       Error;          // <> 0 indica erro
    int16_t       Channel;        // Número do canal
    int16_t       iShuntCal;      // Opção de conexão do shunt cal
                                // 0: conecta no GND
    int16_t       rbConnect;      // Comando de conexão do shunt cal
                                // 0: desconecta em todos os canais
                                // 1: conecta no canal selecionado
} TpPcDLGShuntCal, *PTpPcDLGShuntCal;
```

Salva a Configuração do DLG Motherboard(pcDLGSaveSetup)

Propósito:	Esta mensagem é utilizada pela aplicação do microcomputador para salvar a configuração dos canais dos condicionadores de sinais na memória não volátil do equipamento.
Formato da Mensagem:	TpPcDLGGeneric
Packet Code:	pcDLGSaveSetup
Formato da Resposta:	TpPcDLGGenericResp

Packet Code:	pcDLGSaveSetup
Observações:	As configurações dos condicionadores efetuadas pelas mensagens anteriores ficam na memória RAM. Com isso as modificações realizadas são perdidas quando o equipamento é desligado, Para que elas sejam salvas na memória não volátil deve-se utilizar esta mensagem.

Carrega a Configuração do DLG Motherboard (pcDLGLoadSetup)

Propósito:	Esta mensagem é utilizada pela aplicação do microcomputador para carregar a configuração dos canais dos condicionadores de sinais a partir da memória não volátil do equipamento.
Formato da Mensagem:	TpPcDLGGeneric
Packet Code:	pcDLGLoadSetup
Formato da Resposta:	TpPcDLGGenericResp
Packet Code:	pcDLGLoadSetup
Observações:	

Carrega a Configuração Padrão do DLG Motherboard (pcDLGLoadDefault)

Propósito:	Esta mensagem é utilizada pela aplicação do microcomputador para salvar a configuração dos canais dos condicionadores de sinais na memória não volátil do equipamento.
Formato da Mensagem:	TpPcDLGGeneric
Packet Code:	pcDLGLoadDefault
Formato da Resposta:	TpPcDLGGenericResp
Packet Code:	pcDLGLoadDefault
Observações:	

Carrega a Configuração Padrão de Todas as Configurações do DLG Motherboard e Canais Analógicos (pcDLGResetFlash)

Propósito:	Esta mensagem é utilizada pela aplicação do microcomputador para carregar
-------------------	---

	as configurações de fábrica na memória não volátil
Formato da Mensagem:	TpPcDLGGeneric
Packet Code:	pcDLGResetFlash
Formato da Resposta:	TpPcDLGGenericResp
Packet Code:	pcDLGResetFlash
Observações:	

Salva a Configuração de um Canal Analógico do DLG (pcDLGSaveSetup)

Propósito:	Esta mensagem é utilizada pela aplicação do microcomputador para salvar a configuração dos canais dos condicionadores de sinais na memória não volátil do equipamento.
Formato da Mensagem:	TpPcDLGGeneric
Packet Code:	pcDLGSaveSetup
Formato da Resposta:	TpPcDLGGenericResp
Packet Code:	pcDLGSaveSetup
Observações:	As configurações dos condicionadores efetuadas pelas mensagens anteriores ficam na memória RAM. Com isso as modificações realizadas são perdidas quando o equipamento é desligado, Para que elas sejam salvas na memória não volátil deve-se utilizar esta mensagem.

Carrega a Configuração de um Canal Analógico do DLG (pcDLGLoadSetup)

Propósito:	Esta mensagem é utilizada pela aplicação do microcomputador para carregar a configuração dos canais dos condicionadores de sinais a partir da memória não volátil do equipamento.
Formato da Mensagem:	TpPcDLGGeneric
Packet Code:	pcDLGLoadSetup
Formato da Resposta:	TpPcDLGGenericResp
Packet Code:	pcDLGLoadSetup
Observações:	

MENSAGENS PARA A CONFIGURAÇÃO DA OPERAÇÃO DO COLETOR DE DADOS

Neste tópico serão descritos os formatos das mensagens utilizadas na configuração da operação do coletor de dados.

Mensagem de Escrita da Configuração das Entradas Analógicas (pcDLGSetAiSetup)

Propósito:	A aplicação do microcomputador utiliza esta mensagem para passar a configuração das entradas analógicas a ser utilizada na aquisição de sinais programada no coletor de dados.
Formato da Mensagem:	TpPcDLGSetAiSetup
Packet Code:	pcDLGSetAiSetup
Formato da Resposta:	TpPcDLGGenericResp
Packet Code:	pcDLGSetAiSetupR
Observações:	Os campos sName e sUnit da descrição do canal são strings do Pascal. O primeiro byte desses campos correspondem ao tamanho da string.

```
//----- configuração do canal de entrada analógica --
typedef struct _TpAiDataRec {
    char_t          sName[16];    // Nome do sinal
    char_t          sUnit[8];     // Unidade de engenharia
    float32_t       HiLim;        // Limite superior da escala em
unidade de engenharia
    float32_t       LoLim;        // Limite inferior da escala em
unidade de engenharia
    uint8_t         Used;         // <> 0 indica canal habilitado para
aquisição
    uint16_t        Reservado;
} TpAiDataRec, *PTpAiDataRec;

//----- Tipo para a mensagem pcDLGSetAiSetup --
typedef struct _TpPcDLGSetAiSetup {
    uint16_t        PacketCode;   // Código do pacote
    uint16_t        Reserved;     // reservado
    uint16_t        Error;        // <> 0 indica erro
    TpAiDataRec     AiData[4];    // Configuração das entradas analógicas
} TpPcDLGSetAiSetup, *PTpPcDLGSetAiSetup;
```

Mensagem de Leitura da Configuração das Entradas Analógicas

(pcDLGGetAiSetup)

Propósito:	A aplicação do microcomputador utiliza esta mensagem para ler a configuração das entradas analógicas a ser gravada nos arquivos de série temporal durante a aquisição de sinais.
Formato da Mensagem:	TpPcDLGGeneric
Packet Code:	pcDLGSetAiSetup
Formato da Resposta:	TpPcDLGGenericResp
Packet Code:	pcDLGSetAiSetupR
Observações:	Os campos sName e sUnit da descrição do canal são strings do Pascal. O primeiro byte desses campos correspondem ao tamanho da string.

Mensagem de Leitura da Programação da Aquisição de Sinais do Coletor de Dados (pcDLGGetProgDLog)

Propósito:	A aplicação do microcomputador utiliza esta mensagem para ler a configuração da programação aquisição de sinais do coletor de dados.
Formato da Mensagem:	TpPcDLGGeneric
Packet Code:	pcDLGGetProgDLog
Formato da Resposta:	TpPcDLGGetProgDLogR
Packet Code:	pcDLGGetProgDLogR
Observações:	Os campos SampleFreq, ClockDiv, Period e PreScSw são análogos aos da mensagem pcDLGAcqSetup. Eles definem a frequência de amostragem dos sinais. Os canais a serem aqisitados são especificados pcDLGSetAiSetup. Os dados amostrados são gravados em arquivo de série temporal compatível com o formato TEM do AqDados. A duração de cada arquivo de série temporal é dado pelo parâmetro Duration. A duração poderá ser menor se o trigger de parada estiver habilitado. O campo Mode define o modo de gravação dos pacotes (arquivos): ☐ 0: Continuous Grava o arquivo de série temporal na duração especificada e inicia a próxima gravação imediatamente. Neste modo o trigger é ignorado. ☐ 1: Continuous at Programmed Time Análogo ao modo Continuous com a diferença que neste modo a aquisição do primeiro pacote só se inicia na data e hora especificada em dtStart. Neste modo o trigger é ignorado. ☐ 2: Sequential Grava o arquivo de série temporal na duração especificada e inicia a próxima gravação respeitando o intervalo entre pacotes especificado no parâmetro IntPacket. ☐ 3: Sequential at Programmed Time

Análogo ao modo Sequential com a diferença que neste modo a aquisição do primeiro pacote só se inicia na data e hora especificada em dtStart.

O nome do arquivo de série temporal é definido pelo prefixo fnPrefix, pela data e hora atual e pelo parâmetro FormatFN:

- ☐ 0: YYYY-MM-DD-hh-mm.TEM
- ☐ 1: YYYY-MM-DD-hh-mm-ss.TEM
- ☐ 2: YYYY-MM-DD\YYYY-MM-DD-hh-mm.TEM
- ☐ 3: YYYY-MM-DD\YYYY-MM-DD-hh-mm-ss.TEM
- ☐ 4: YYYY-MM\YYYY-MM-DD-hh-mm.TEM
- ☐ 5: YYYY-MM\YYYY-MM-DD-hh-mm-ss.TEM

Os campos FormatFN e fnPrefix são reservados para uso futuro. O nome dos arquivos deve utilizar o seguinte formato:

- ☐ YYYYMMDD\hh-mm-ss.TEM
-

```
//----- Configuração do trigger início / término --
typedef struct _TpTrigSetup {
    uint8_t      Enabled;          // <> 0 habilita trigger
    uint8_t      Channel;          // Canal A/D de trigger
    int16_t      LevelAD;          // Nível do trigger (em unidade do A/D)
    float32_t    LevelEU;          // Nível do trigger (em unidade de
    engenharia)
    uint16_t      Edge;            // Borda do trigger: 0 (descida) e 1 (subida)
    uint16_t      Time;            // Tempo de pré-trigger / pós-trigger (em ms)
} TpTrigSetup, *PTpTrigSetup;

//----- Configuração da aquisição no modo coletor de dados --
typedef struct _TpProgDLog {
    int32_t      Duration;          // Duração do pacote (ensaio) em segundos
    float32_t    SampleFreq;        // Frequência de amostragem em Hz
    uint16_t     ClockDiv;          // Clock divider do SSC clock mode
    register (0..4095) (ignorado)
    uint16_t     Period;            // Receive clock divider do SSC
    clock mode register
                                // (0..255) (ignorado)
    uint16_t     PreScSw;           // Prescaler de software (ignorado)
    int32_t      IntPacket;         // Intervalo entre pacotes (em segundos)
    uint16_t     Mode;              // Modo da aquisição de sinais
                                // 0 = Contínuo
                                // 1 = Contínuo Programado
                                // 2 = Sequential
                                // 3 = Sequencial Programado
    uint16_t     FormatFN;          // Formato para o nome do arquivo
    TpDLGDateTime dtStart;          // Data/hora para iniciar a
    aquisição
                                // (no modo Programmed)
    TpTrigSetup  StartTrig;         // Configuração do trigger de início de aquisição
    TpTrigSetup  StopTrig;          // Configuração do trigger de fim de aquisição
    char_t       fnPrefix[16];      // Prefixo para o nome do arquivo
} TpProgDLog, *PTpProgDLog;

//----- Tipo para a mensagem pcDLGGetProgDLog --
typedef struct _TpPcDLGGetProgDLog {
    uint16_t     PacketCode;        // Código do pacote
    uint16_t     Reserved;          // reservado
    uint16_t     Error;             // <> 0 indica erro
    TpProgDLog   ProgDLog;          // Programação da aquisição de sinais
                                // do coletor de dados
} TpPcDLGGetProgDLog, *PTpPcDLGGetProgDLog;
```

Mensagem de Programação da Aquisição de Sinais do Coletor de Dados (pcDLGSetProgDLog)

Propósito:	A aplicação do microcomputador utiliza esta mensagem para programar a aquisição de sinais do coletor de dados.
Formato da Mensagem:	TpPcDLGSetProgDLog
Packet Code:	pcDLGSetProgDLog
Formato da Resposta:	TpPcDLGSetProgDLog
Packet Code:	pcDLGSetProgDLogR
Observações:	

```
typedef struct _TpPcDLGSetProgDLog {
    uint16_t      PacketCode;  // Código do pacote
    uint16_t      Reserved;    // reservado
    TpProgDLog    ProgDLog;    // Programação da aquisição de sinais
                                // do coletor de dados
} TpPcDLGSetProgDLog, *PTpPcDLGSetProgDLog;

typedef struct _TpPcDLGSetProgDLogR {
    uint16_t      PacketCode;  // Código do pacote
    uint16_t      Reserved;    // reservado
    float32_t     SampleFreq;  // Programação da aquisição de sinais
                                // do coletor de dados
} TpPcDLGSetProgDLogR, *PTpPcDLGSetProgDLogR;
```

Mensagem de Comando para o Coletor de Dados (pcDLGCommandDLog)

Propósito:	A aplicação do microcomputador utiliza esta mensagem para programar a aquisição de sinais do coletor de dados.
Formato da Mensagem:	TpPcDLGSetProgDLog
Packet Code:	pcDLGGetGPS_Setup
Formato da Resposta:	TpPcDLGCommandDLogR
Packet Code:	pcDLGSetProgDLogR

Observações:

O campo Command define o comando a ser executado pelo coletor de dados:

0: Get Status

Informa o status da operação do coletor de dados.

1: Start

Inicia a operação do coletor de dados conforme a programação previamente enviada. Se o DLG estiver com a aquisição de sinais em tempo real ativa (veja as mensagens pcAcqSetup e pcAcqStart) e a frequência de amostragem configurada para operação no modo coletor de dados for diferente da aquisição em tempo real, o comando Start deve ser rejeitado (código de erro diferente de zero).

2: Stop

Finaliza a operação do coletor de dados. A coleta que eventualmente esteja em andamento deve ser finaliza e o arquivo fechado.

```
//----- Tipo para a mensagem pcDLGCommandDLog --
typedef struct _TpPcDLGCommandDLoog {
    uint16_t      PacketCode;    // Código do pacote
    uint16_t      Reserved;      // reservado
    uint16_t      Command;       // Código do comando
} TpPcDLGCommandDLog, *PTpPcDLGCommandDLog;

//----- Tipo para a mensagem pcDLGCommandDLogR --
typedef struct _TpPcDLGCommandDLogR {
    uint16_t      PacketCode;    // Código do pacote
    uint16_t      Reserved;      // reservado
    uint16_t      Error;         // <> 0 indica erro
    uint16_t      Status;        // 0: coletor de dados parado
                                // 1: aquisição de dados habilitada
} TpPcDLGCommandDLogR, *PTpPcDLGCommandDLogR;
```

Mensagem de Escrita da Configuração da Operação do GPS (pcDLGSetGPS_Setup)

Propósito:	A aplicação do microcomputador utiliza esta mensagem para programar a aquisição de sinais do coletor de dados.
Formato da Mensagem:	TpPcDLGSetGPS_Setup
Packet Code:	pcDLGSetGPS_Setup
Formato da Resposta:	TpPcDLGGenericResp
Packet Code:	pcDLGSetGPS_SetupR
Observações:	Os dados de posicionamento e tempo do GPS são gravados quando a aquisição de dados está ativa e no intervalo de tempo especificado no parâmetro Interval. O intervalo é especificado em segundos. A habilitação da gravação do arquivo de dados do GPS é definida no parâmetro Enable.

```
//----- Tipo para a mensagem pcDLGSetGPS_Setup --
typedef struct _TpPcDLGSetGPS_Setup {
    uint16_t      PacketCode;    // Código do pacote
    uint16_t      Reserved;      // reservado
    uint16_t      Error;         // <> 0 indica erro
    uint16_t      Enable;        // <> 0 habilita gravação do
arquivo de dados do GPS
    uint16_t      Interval;      // Intervalo (em segundos) de
gravação de registros
                                // do arquivo de dados do GPS
} TpPcDLGSet GPS_Setup, *PTpPcDLGSet GPS_Setup;
```

Mensagem de Leitura da Configuração da Operação do GPS (pcDLGGetGPS_Setup)

Propósito:	A aplicação do microcomputador utiliza esta mensagem para ler a configuração da operação do GPS.
Formato da Mensagem:	TpPcDLGGeneric
Packet Code:	pcDLGGetGPS_Setup
Formato da Resposta:	TpPcDLGGetGPS_SetupR
Packet Code:	pcDLGGetGPS_SetupR
Observações:	

```
//----- Tipo para a mensagem pcDLGGetGPS_Setup --
typedef struct _TpPcDLGGetGPS_Setup {
    uint16_t      PacketCode;    // Código do pacote
    uint16_t      Reserved;      // reservado
    uint16_t      Error;         // <> 0 indica erro
    uint16_t      Enable;        // <> 0 habilita gravação do
arquivo de dados do GPS
    uint16_t      Interval;      // Intervalo (em segundos) de
gravação de registros
                                // do arquivo de dados do GPS
} TpPcDLGGetGPS_Setup, *PTpPcDLGGetGPS_Setup;
```

Revisões

REV1

- mudança em TpPcDLGSetInfo: campo "Notch" → "Chan_Opt" (informação de canais e se sistema tem GPS, Rainflow, etc)
- mudança em protocolo de comunicação com AVR.
- Mudança da frequência de corte do filtro
- Alteração em setinfochan do protocolo UDP
- Setinfo UDP: NIP2 → FPGA_Version

REV2

- Mudanças em Setinfo chan e getinfochan

REV3

- Mudanças na estrutura TpGetInfoChan: VersionP1 e localização de dtAdjust2k.